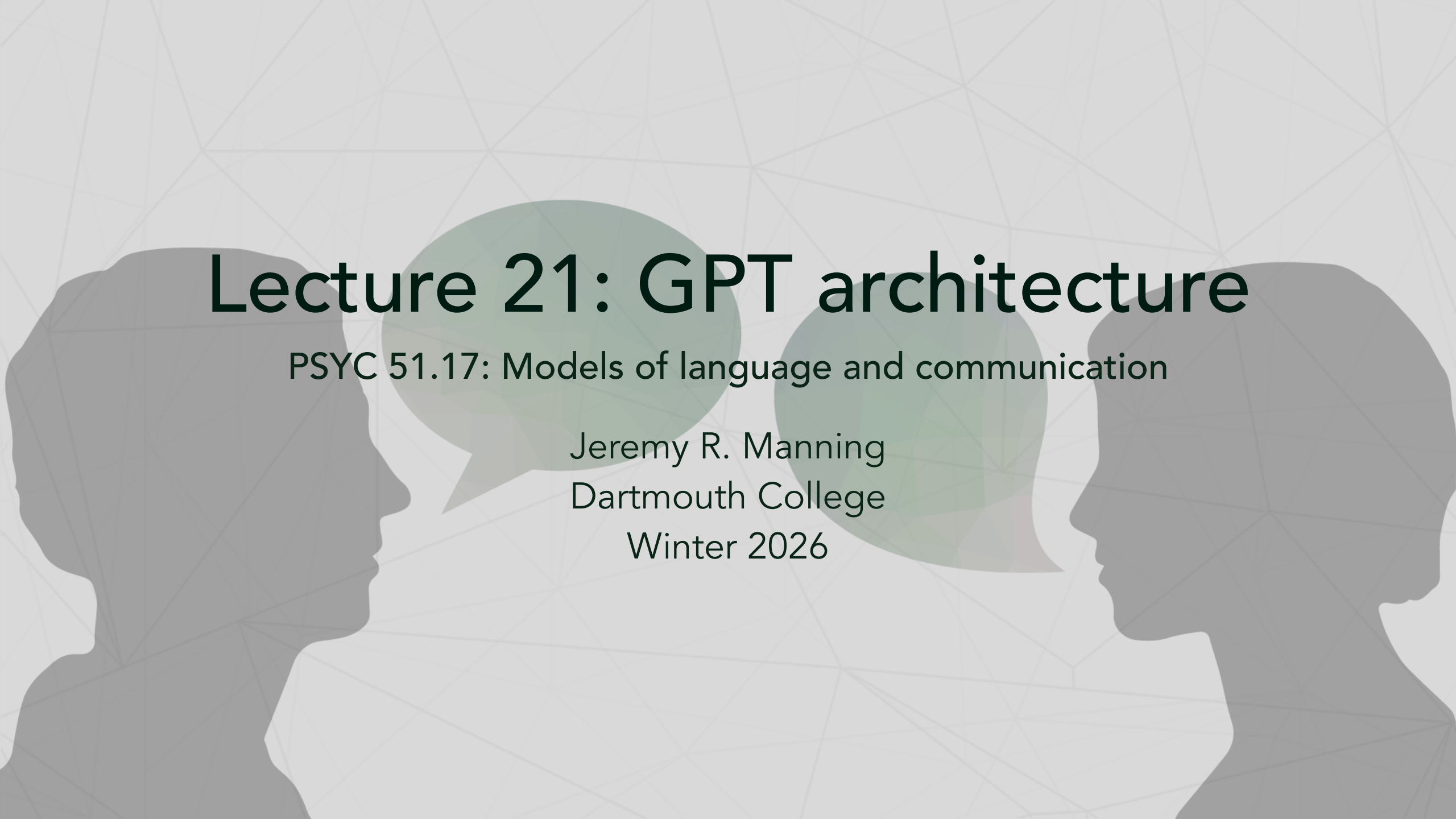


Lecture 21: GPT architecture

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain the **generative pre-training** paradigm and why it was revolutionary
2. Evaluate the **ethical implications** of GPT-1's training data (BooksCorpus)
3. Distinguish between **pre-training** and **fine-tuning** stages
4. Compare GPT's **autoregressive** approach with BERT's **bidirectional** approach
5. Identify how the decoder stack has evolved from GPT-1 to open-weight LLMs (2025)

Announcements

Assignment 4 due this week

Assignment 4 (Customer Service Chatbot) is due **February 16, 11:59 PM EST**.

Final project and optional assignment

- **Final Project** has been released! Due **March 9, 11:59 PM EST**
- **Assignment 5** (Build GPT) is now available as **optional extra credit**

The generative pre-training paradigm

Radford et al. (2018): 'Improving Language Understanding by Generative Pre-Training'

GPT introduced a two-stage approach that changed NLP forever:

1. **Pre-train** on massive unlabeled text (unsupervised)
2. **Fine-tune** on specific downstream tasks (supervised)

Why this was revolutionary

- **Before GPT**: Train task-specific models from scratch, needing large labeled datasets for each task
- **After GPT**: Learn general language representations once, then adapt cheaply to any task
- This is **transfer learning** for NLP — the same idea that transformed computer vision with ImageNet

GPT-1 specifications

Model architecture

Component	Value
Transformer layers	12
Hidden size (d_model)	768
Attention heads	12
Max sequence length	512 tokens
Vocabulary size	40,000 (BPE)
Feed-forward size	3,072 (4 × 768)
Total parameters	~117 million

Training details

Detail	Value
Training data	BooksCorpus (~7,000 books)

The BooksCorpus controversy

Where did GPT-1's training data come from?

GPT-1 was trained on **BooksCorpus** — approximately 7,000 unpublished books scraped from Smashwords.com, a self-publishing platform. The authors were never asked for consent, and the dataset was **taken offline in 2020** after complaints from writers who discovered their work had been used.

Questions to consider

BooksCorpus was just the beginning. Later datasets — **Books3** (196,000 books), **The Pile**, **Common Crawl** — sparked lawsuits and a global debate about data rights. If your unpublished novel helped train GPT-1, should you have been informed? Compensated? Given the right to opt out?

This tension between **data access** (enabling research) and **creator rights** (protecting authors) remains unresolved in 2026.

Pre-training objective

Next token prediction

Maximize the likelihood of each token given all preceding tokens:

$$\mathcal{L}_{\text{pre-train}} = \sum_{i=1}^{N} \log P(t_i \mid t_1, t_2, \ldots, t_{i-1}; \Theta)$$

This is simply **next token prediction** over a large corpus. No labels needed. The causal attention mask (Lecture 15) ensures each token only attends to previous positions, enabling parallel training over all positions simultaneously.

Stage 2: Fine-tuning

Adapting to downstream tasks

After pre-training, GPT is adapted to specific tasks by:

1. **Reformatting** task inputs with special delimiter tokens
2. **Adding** a small classification head (linear layer) on top
3. **Training** with both task loss and a language modeling auxiliary loss

$$\mathcal{L}_{\text{fine-tune}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{LM}}$$

The auxiliary LM loss ($\lambda = 0.5$) improves generalization and speeds convergence.

Task formatting

All tasks become text completion: `[START] text [DELIM]` → predict label. Classification, entailment, similarity, and QA all use the same architecture with different input formats. This unifying insight — every NLP task as text completion — is why pre-training transfers so effectively.

Fine-tuning example: sentiment classification

Classifying movie reviews

```
# Format input for GPT
text = "This movie was absolutely fantastic!"
formatted = f"[START] {text} [DELIM]"

# Tokenize and pass through GPT
token_ids = tokenizer.encode(formatted)
hidden_states = gpt_model(token_ids)  # (1, seq_len, 768)

# Take hidden state at the last (DELIM) position
final_hidden = hidden_states[0, -1, :]  # (768,)

# Pass through classification head
classification_head = nn.Linear(768, 2)  # 2 classes
logits = classification_head(final_hidden)  # [pos_score,
neg_score]

```

continued...

Fine-tuning example: sentiment classification

Classifying movie reviews

```
# Compute loss
loss = cross_entropy(logits, label_id)  # label_id = 0 (positive)
```

...continued

GPT-1 results

Performance on standard benchmarks

Task	Previous SOTA	GPT-1
Question answering	86.7	**88.1**
Semantic similarity	85.0	**85.8**
Text classification	93.0	**94.2**
Natural language inference	80.6	**82.1**

GPT-1 achieved state-of-the-art on **9 out of 12** benchmark tasks.

The key finding

The largest improvements came on tasks with **less training data**. Pre-training provided a strong prior that compensated for limited labeled examples — exactly the promise of transfer learning.

Zero-shot and few-shot learning

Learning without (much) fine-tuning

- **Zero-shot**: No task-specific examples. The model must generalize purely from its pre-training.
- **Few-shot**: A small number of examples (1–10) provided as context in the prompt.
- **Fine-tuning**: Full supervised training on a task-specific dataset.

GPT-1's limitation

GPT-1's zero-shot performance was **weak**. The model had learned rich language representations but struggled to apply them without explicit task formatting. This motivated the development of GPT-2 and GPT-3, which aimed to make models that could perform tasks *without* fine-tuning.

Weight tying

Sharing parameters between input and output layers

Modern GPT models share weights between the **token embedding** layer and the **output (lm_head)** layer (Press & Wolf, 2017):

```
self.token_embed = nn.Embedding(vocab_size, d_model)
self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
self.lm_head.weight = self.token_embed.weight  # Tied!
```

Why this works

Both layers map between **token space** and **embedding space** — just in opposite directions. The embedding layer converts token IDs → vectors; the lm_head converts vectors → token probabilities. Tying them forces consistent representations and **saves ~20% of total parameters** in vocabulary-heavy models. Used in GPT-2, LLaMA, and most modern LLMs.

The modern decoder stack (2024)

Every component has been upgraded since GPT-1

Component	GPT-1 (2018)	Modern LLMs (2024)	Why the change
Normalization	LayerNorm	RMSNorm	10–15% faster, no mean computation
Position encoding	Learned absolute	RoPE	Extrapolates to unseen lengths
Activation	GELU	SwiGLU	~1% better across benchmarks
Attention	Multi-head (MHA)	Grouped-query (GQA)	2× faster inference, same quality

The takeaway

The *conceptual* architecture is the same: token embeddings → causal attention → FFN → output head. But every piece has been systematically optimized. Modern LLMs like LLaMA 3, Gemma 2, and Mistral all use this upgraded stack.

Open-weight decoders: the LLaMA revolution

From closed to open

For years, cutting-edge decoders were **closed** — GPT-3 and GPT-4 were API-only. Meta's LLaMA (Feb 2023) changed everything by releasing model weights publicly.

Model	Date	Sizes	Key contribution
LLaMA 1	Feb 2023	7–65B	Leaked, then open. Proved open models competitive.
LLaMA 2	Jul 2023	7–70B	Official open release with commercial license
Mistral 7B	Dec 2023	7B	Small open model matching LLaMA 2 13B
LLaMA 3	Apr 2024	8–405B	Matched GPT-4 class on many benchmarks
LLaMA 4	Apr 2025	MoE	Mixture-of-experts architecture

Impact

Open weights enabled academic research, spawned thousands of fine-tuned variants, and **democratized**

Test-time compute and inference scaling

A new scaling paradigm

Traditional scaling: more parameters + more training data. **Inference scaling**: spend more compute at *test time* by letting the model "think longer."

How it works

Approach	Example	Mechanism
Chain-of-thought	GPT-4, Claude	Prompting the model to reason step-by-step
Thinking tokens	OpenAI o1/o3	Model generates internal reasoning traces before answering
Open reasoning	DeepSeek-R1	Open-source reasoning model with visible thinking process
Search + verify	AlphaProof	Generate candidates, verify with external tools

Why this matters

Inference scaling means you don't need to retrain a model to make it better at hard problems — just give it more time to think. This shifts the cost curve: training is fixed, but inference quality scales with compute budget. We'll explore reasoning models in detail in Lecture 22.

Multi-token prediction

Predicting more than one token at a time (Meta/FAIR, 2024)

Standard GPT predicts one token ahead. Multi-token prediction trains the model to predict the **next 2–4 tokens simultaneously** using independent output heads sharing the same backbone.

Why this matters

Benefit	Explanation
Better representations	Forces the model to plan ahead, not just match local patterns
Faster inference	Can decode 2–4× faster with speculative decoding
Stronger coding	12% improvement on code generation (HumanEval), where planning matters most

The connection to how humans process language

Humans don't process language one word at a time — we predict upcoming words *in chunks*.

Hybrid architectures: attention meets state-space models

Not everything needs to be a transformer

Jamba (AI21, 2024) interleaves Transformer attention layers with Mamba state-space layers:

- **Attention layers**: Good at precise retrieval ("what was the third item?")
- **Mamba layers**: Good at long-range compression and fast inference ($O(n)$ vs $O(n^2)$)
- **Hybrid**: Gets the best of both — 256K context at 3× the throughput of pure Transformers

Questions to consider

The Transformer has dominated since 2017. But Jamba, Mamba-2, and RWKV suggest that the optimal architecture may be a *hybrid*. What does it mean that different computational primitives (attention vs. recurrence) excel at different aspects of language?

Limitations of GPT-1 and the path forward

What GPT-1 could not do well

- **Weak zero-shot performance**: Required fine-tuning for each new task
- **Small model**: 117M parameters (small by modern standards)
- **Limited data**: Trained on ~5B tokens from BooksCorpus only
- **Short context**: Maximum sequence length of 512 tokens
- **Hallucinations**: Confidently stated incorrect facts

The path forward

Each limitation suggested a clear direction: bigger models, more data, longer contexts, better training. GPT-2 and GPT-3 would systematically address these limitations through **scale** — but as we'll see in the next lecture, scale alone brings its own surprises and controversies.

Discussion

Questions to consider

1. **Training data ethics:** GPT-1 trained on books scraped without consent; later models used even larger datasets with similar issues. Where should the line be drawn between research progress and creator rights? Is opt-out sufficient, or should training require opt-in?
2. **Open vs closed:** LLaMA democratized decoder research but also enabled misuse (fine-tuning for harmful purposes). Is openness a net positive? Should frontier models be open?
3. **The generation advantage:** GPT can *generate* text, BERT cannot. Is generation a prerequisite for understanding? Can you truly understand language if you can't produce it?
4. **Inference scaling:** If models can "think harder" by spending more

Further reading

Further reading

Radford et al. (2018) "Improving Language Understanding by Generative Pre-Training" — The original GPT paper.

Radford et al. (2019) "Language Models are Unsupervised Multitask Learners" — GPT-2: larger models, zero-shot transfer.

Touvron et al. (2023, *arXiv*) "LLaMA: Open and Efficient Foundation Language Models" — The paper that opened decoder research.

Press & Wolf (2017, *EACL*) "Using the Output Embedding to Improve Language Models" — Weight tying between input and output embeddings.

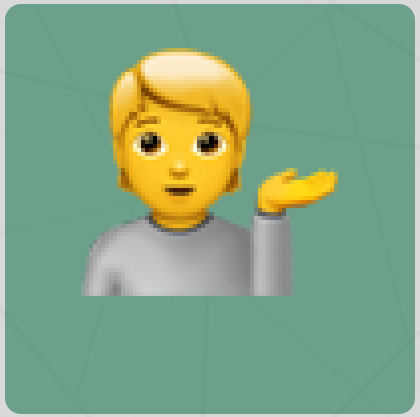
Gloeckle et al. (2024, *arXiv*) "Better & Faster Large Language Models via Multi-token Prediction" — Meta/FAIR multi-token prediction.

Lieber et al. (2024, *arXiv*) "Jamba: A Hybrid Transformer-Mamba Language Model" — Hybrid attention + SSM architecture.

Questions?

Email

Discord



Office hours

Up next...

Scaling up: from GPT-2 to GPT-4, emergent abilities, reasoning, and the path to ChatGPT